

Automation Framework Master Notes

Goal

Build a professional enterprise-level automation framework using:

- Java
 - Selenium
 - TestNG
 - Maven
 - Jenkins
 - REST Assured
 - GitHub
 - Apache POI
 - Logging
 - Reporting
-

1. Framework Objectives

The framework should be:

- Maintainable
 - Reusable
 - Scalable
 - Stable
 - CI/CD ready
 - Cross-browser compatible
 - Easy to debug
 - Easy to execute
 - Professional for enterprise environments
-

2. Recommended Framework Structure

src/test/java

```
|
|— tests
|— pages
|— locators
|— utilities
|— reports
```

- api
- data
- base
- listeners

3. Utilities Classes

Purpose

Utility classes help avoid duplicate code.

Utilities To Build

Screenshot Utility

Purpose:

- Capture screenshots
- Capture screenshots on pass/fail
- Save screenshots to framework folder
- Attach screenshots to reports

Methods:

Screenshot.[takeScreenshot](#)(driver)

Wait Utility

Purpose:

- Handle synchronization
- Prevent flaky tests

Avoid:

[Thread](#).[sleep](#)()

Use:

WebDriverWait
ExpectedConditions

Methods:

```
WaitUtils.waitForElementVisible()  
WaitUtils.waitForClickable()
```

Scroll Utility

Purpose:

- Scroll elements into view
- Scroll page up/down

Methods:

```
ScrollUtils.scrollToElement()  
ScrollUtils.scrollToBottom()
```

Click Utility

Purpose:

- Handle normal clicks
- Handle JavaScript clicks
- Reduce click failures

Methods:

```
ClickUtils.click()  
ClickUtils.jsClick()
```

Browser Setup Utility

Purpose:

- Launch browsers centrally
- Reusable browser configuration

Supported Browsers:

- Chrome
- Firefox
- Edge

Methods:

```
DriverFactory.getDriver()
```

Report Utility

Purpose:

- Generate reports automatically
- Add screenshots to reports
- Create evidence documents

Tools:

- Extent Reports
- Apache POI
- Allure Reports

Current Goal:

- Word report with screenshots

Future Goal:

- HTML reports
-

4. Data-Driven Testing

Purpose

Run tests with multiple datasets.

Instead of:

```
login("admin", "123")
```

Use:

- Excel
 - CSV
 - JSON
 - TestNG DataProvider
-

TestNG DataProvider

Example:

```
@DataProvider(name = "loginData")
```

Benefits:

- Reusable test data
 - Multiple executions
 - Better coverage
-

Excel Data

Tool:

- Apache POI

Use cases:

- Login data
- Search data
- API payloads

Future utility:

`ExcelUtils.readData()`

CSV Data

Use cases:

- Lightweight datasets
 - Large bulk data
-

JSON Data

Use cases:

- API payloads
 - Environment configuration
 - Dynamic test data
-

5. Explicit Waits

Important Rule

Never rely heavily on:

`Thread.sleep()`

Why?

- Slows tests
- Causes instability
- Not scalable

Use:

WebDriverWait
ExpectedConditions

Benefits:

- Stable automation
 - Faster execution
 - Better synchronization
-

6. Logging

Avoid

```
System.out.println()
```

Use Professional Logging

Tools:

- Log4j
- SLF4J

Benefits:

- Cleaner logs
- Better debugging
- Professional framework
- Easier Jenkins troubleshooting

Example:

```
logger.info("Login successful")  
logger.error("Login failed")
```

7. CI/CD Integration

Goal

Automation should run automatically.

CI/CD Tools

Jenkins

Purpose:

- Run automation suites automatically
- Schedule nightly runs
- Execute smoke tests
- Generate reports

Examples:

- Nightly regression
 - Smoke suite after deployment
-

GitHub Actions

Purpose:

- Cloud CI/CD
 - Run tests after code push
-

Azure DevOps

Purpose:

- Enterprise pipelines
 - Build + Release automation
-

CI/CD Pipeline Example

Developer Pushes Code



GitHub



Jenkins Triggered



Maven Build
↓
Run TestNG Suite
↓
Generate Reports
↓
Email Results

8. API + UI Hybrid Framework

Goal

Build a framework that tests:

- UI
 - API
 - Database
-

UI Automation

Tool:

- Selenium

Used for:

- Login
 - Cart
 - Checkout
 - UI validations
-

API Automation

Tool:

- REST Assured

Used for:

- GET requests
- POST requests
- PUT requests
- DELETE requests

- JSON validations

Example:

```
given()  
.when()  
.get("/users")  
.then()  
.statusCode(200);
```

Database Validation

Future Goal: Validate database records.

Tools:

- JDBC
- SQL

Example:

```
SELECT * FROM USERS
```

9. Reporting Goals

Current

- Word reports
- Screenshots

Future

- Extent Reports
 - Allure Reports
 - Jenkins reports
 - Email reports
-

10. Maven Integration

Purpose

Manage:

- Dependencies

- Builds
- Test execution

Commands:

```
mvn clean  
mvn test  
mvn install
```

11. TestNG Integration

Purpose

Manage:

- Test execution
- Assertions
- Parallel execution
- DataProviders
- Reports

Important Annotations:

```
@BeforeMethod  
@Test  
@AfterMethod  
@DataProvider
```

12. Future Enhancements

Add Later

- Parallel execution
- Docker
- Selenium Grid
- Cloud execution
- Retry mechanism
- Headless execution
- Email automation
- Cross-browser execution
- Mobile testing
- BDD Cucumber integration

13. Enterprise Framework Vision

Final framework should support:

Java
Selenium
TestNG
Maven
REST Assured
Jenkins
GitHub
Logging
Reporting
Data-Driven Testing
CI/CD
API + UI Testing
Database Validation

14. Learning Path

Phase 1

- Selenium
- Java
- TestNG
- Maven

Phase 2

- Framework utilities
- Reporting
- Screenshots
- Logging

Phase 3

- REST Assured
- API testing
- Hybrid framework

Phase 4

- Jenkins
- CI/CD

- GitHub Actions

Phase 5

- Docker
 - Grid
 - Cloud execution
-

15. Golden Rules

- Keep code reusable
- Avoid duplication
- Use explicit waits
- Never hardcode test data
- Capture screenshots on failures
- Keep framework modular
- Use proper naming conventions
- Push code to GitHub regularly
- Keep framework CI/CD ready
- Build for scalability